

Dash Basics: HTML and Core Components

What I'm Doing

In this lab, I **build an interactive dashboard** using Dash and Plotly that **visualises airline flight data**. Specifically, I **create a single-page web application displaying a pie chart** that shows the proportion of flights grouped by distance intervals (every 250 miles), using HTML components for the layout and a core graph component for the chart.

1. Setting Up

First, I install the required Python packages. The terminal commands install `packaging` (a dependency), `pandas` for data handling, `dash` for the web framework, and `plotly` for charting.

```
%pip install packaging
%pip install pandas dash
%pip install httpx dash plotly
```

With the dependencies ready, I create a new Python script called `dash_basics.py` and open a terminal in the IDE to run it later.

2. Getting to Know the Dataset

The dataset used is the **Airline Reporting Carrier On-Time Performance** data. It contains detailed information about domestic flights, including arrival/departure times, delays, distances, and more. For this lab, I work with a random sample of 500 records to keep things manageable.

3. Data Preparation

I import the necessary libraries, read the dataset from its remote URL, sample 500 random rows, and create a pie chart showing the proportion of flights by distance group.

```
# Import required packages
import pandas as pd
import plotly.express as px
import dash
from dash import dcc
from dash import html
```

```
# Read the airline data into pandas dataframe
airline_data = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/
Data%20Files/airline_data.csv',
    encoding = "ISO-8859-1",
    dtype={'Div1Airport': str, 'Div1TailNum': str,
```

```

        'Div2Airport': str, 'Div2TailNum': str}}

# Randomly sample 500 data points. Setting the random state to be 42 so that we
get same result.
data = airline_data.sample(n=500, random_state=42)

# Pie Chart Creation
fig = px.pie(data, values='Flights', names='DistanceGroup', title='Distance group
proportion by flights')

```

Result: A `fig` object holding a Plotly pie chart where each slice represents a distance group (intervals of 250 miles), and the size of each slice corresponds to the number of flights in that group.

What's happening here?

- `px.pie()` creates a pie chart from the sampled data.
- `values='Flights'` tells Plotly to use the `Flights` column to size each slice.
- `names='DistanceGroup'` uses the `DistanceGroup` column for the slice labels.
- A random state of 42 ensures reproducible results

4. Building the Dash Application Layout

Now I create the Dash application itself. The dashboard has three visual components:

Component	Dash Tag	Purpose
Title	<code>html.H1()</code>	Application heading
Description	<code>html.P()</code>	Paragraph explaining the chart
Chart	<code>dcc.Graph()</code>	The interactive pie chart

```

# Create a dash application
app = dash.Dash(__name__)

```

`dash.Dash(__name__)` **initialises a new Dash application**. The `__name__` argument tells Dash where to find static assets and is standard practice.

Next, I define the layout. The layout is wrapped in an outer `html.Div()` that holds all child components.

```

# Get the layout of the application and adjust it.
app.layout = html.Div(children=[html.H1('Airline Dashboard',
    style={'textAlign': 'center', 'color': '#503D36', 'font-size': 40}),
    html.P('Proportion of distance group (250 mile distance interval group) by
flights.',
    style={'textAlign': 'center', 'color': '#F57241'}),
    dcc.Graph(figure=fig),
])

```

Breaking down each component:

- **html.H1** - Renders an `<h1>` HTML element. I set the title to 'Airline Dashboard' and style it with centre alignment, dark blue-grey colour (#503D36), and a font size of 40px.
- **html.P** - Renders a `<p>` element. The description explains what the chart shows, styled centre-aligned with an orange colour (#F57241).
- **dcc.Graph** - The core graph component from Dash. I pass the `fig` object (the pie chart created earlier) to the `figure` parameter.

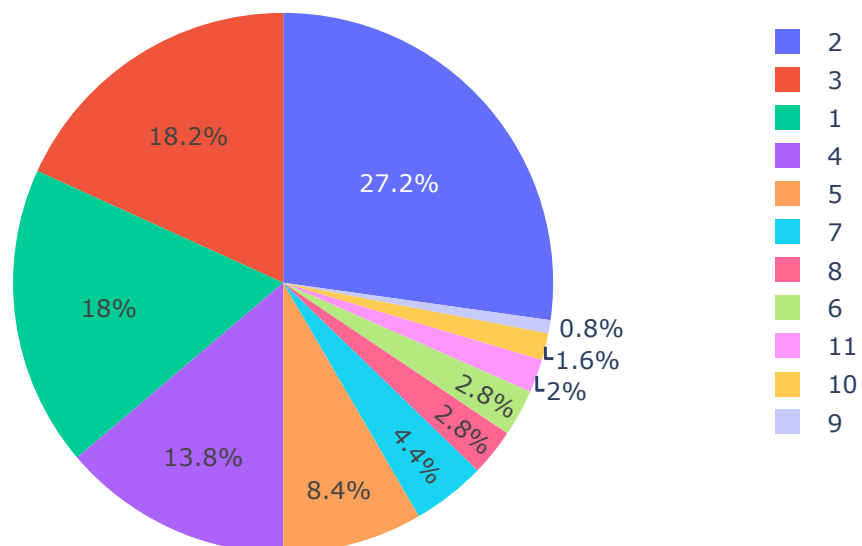
Finally, I add the standard entry point to run the app:

```
# Run the application
if __name__ == '__main__':
    app.run()
```

Airline On-time Performance Dashboard

Proportion of distance group (250 mile distance interval group) by flights.

Distance group proportion by flights



5. Running the Application

(skip this step when running in notebook)

I save the script, then run it from the terminal:

```
python dash_basics.py
```

Result: The Dash development server starts, outputting a local URL (`http://127.0.0.1:8050/`). I note the port number shown in the terminal, click the **Launch Application** button from the IDE's side menu, enter the port number, and the dashboard opens in a new browser tab.

The dashboard displays:

1. **"Airline Dashboard"** as a centred heading
2. A paragraph describing the chart
3. An interactive pie chart showing distance group proportions - I can hover over slices to see exact values

6. Exercise: Practice Tasks

Exercise: Updating the Dashboard Title

The task asks me to **change the title** from "Airline Dashboard" to "Airline On-time Performance Dashboard" and **increase the font size** to 50.

```
app.layout = html.Div(children=[html.H1('Airline On-time Performance Dashboard',
    style={'textAlign': 'center', 'color': '#503D36', 'font-size': 50}),
    html.P('Proportion of distance group (250 mile distance interval group) by
    flights.',
    style={'textAlign': 'center', 'color': '#F57241'}),
    dcc.Graph(figure=fig),
])
```

What changed: I updated the string inside `html.H1()` and bumped `font-size` from 40 to 50.

To see the update, I save the file (`File → Save`), go to the terminal, and restart the app:

```
python dash_basics.py
```

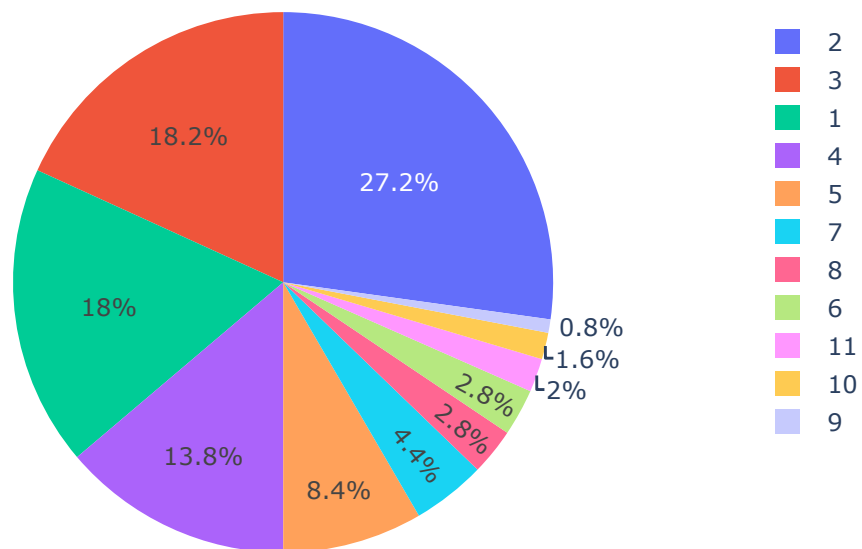
for running **inside notebook**, run this code:

```
if __name__ == '__main__':
    app.run()
```

Airline On-time Performance Dashboard

Proportion of distance group (250 mile distance interval group) by flights.

Distance group proportion by flights



The updated dashboard now shows the longer title in a larger font. To **stop the running application**, I press **Ctrl + C** in the terminal.

That's the full walkthrough. From installing dependencies to deploying a live interactive dashboard, this lab demonstrates how to combine Dash's HTML components with Plotly visualisations to turn raw airline data into a clear, browser-ready flight-distance breakdown.